PROCESS-MEMORY

코드·데이터·힙·스택과 메모리 오류

연관 기출 · 138-2-6, 136-3-6, 136-1-7

한눈에 이해

실행할 명령, 계속 유지할 값, 동적으로 얻은 값, 함수 호출의 임시값은 수명과 관리 방법이 다릅니다.

정의와 핵심 원리

일반적인 프로세스 주소 공간은 실행 코드·정적데이터·동적힙·호출 스택 등 논리 영역으로 설명할 수 있습니다. 실제 실행파일·메모리 매핑은 운영체제·언어·실행 환경에 따라 더 세분됩니다.

• 코드는 명령과읽기전용자료를 두며데이터는 초기화 전역·정적 변수와BSS등을 포함합니다.힙은 동적 할당객체의 수명을 관리하고 스택은 호출별인자·지역변수·복귀정보를 프레임으로 보관합니다.모든 지역변수가반드시 스택에 있는 것은아니며최적화·클로저·코루틴의예외가 있습니다.

• C의malloc은 영역을 할당하고 free는 반환합니다.참조를 잃어반환하지못하면 누수, 반환뒤접근하면 use-after-free, 중복반환은double-free입니다.Java/Python도 불필요한 객체를 컬렉션·캐시·리스너가계속참조하면 GC가 회수하지못합니다.

• 세그멘테이션오류는 허용되지않은메모리 접근에 대한보호위반으로NULL·경계초과·해제후접근등이 원인일수 있습니다.메모리 누수는사용량추세·힙덤프·보유 참조 분석으로, 보호위반은덤프·스택추적·메모리 검사기로원인을 좁힙니다.

구조·비교·판단

구분	핵심 원리	적용·한계
코드·데이터	프로그램·정적수명	권한과 초기화 구분
힙	객체별동적수명	누수·해제후사용
스택	호출 프레임 수명	깊은재귀·범위탈출참조

적용 예제

요청마다 쌓이는 캐시

강사 생성 학습 예제입니다. 공식 기준답안이 아닙니다.

조건: 요청 1회마다1MiB객체를 전역리스트에추가;1000회

1. 요청이 끝나도전역리스트가객체를 참조합니다.
2. 대략1000MiB+객체 관리 비용이 남아압력이증가합니다.
3. 힙덤프의지배객체·참조 경로로 리스트를확인합니다.
4. 필요한 보관기간·항목한도를정하고 수명 종료시에참조를 제거합니다.

결과·해석: GC 호출만 반복하는 대신 도달가능한 불필요 객체의 보유 원인을 수정합니다.

전제·주의: 논리 영역 그림은개념 모델이며 힙·스택의 성장방향을모든 플랫폼의 법칙으로쓰지않습니다.

답안 골격과 혼동 방지

네영역역할·수명 → 호출·할당 흐름 → 누수·보호위반 구분 → 언어별진단 → 재발방지

- GC가 있으면 누수가불가능하다고쓰지않습니다.
- SIGSEGV를 항상 물리 메모리부족으로해석하지 않습니다.

내용을 가리고 회상

1. 힙 객체가 GC대상에 서 제외되는이유는무엇입니까?
2. 스택지역값의 참조를 반환하면 왜위험합니까?
3. 누수와해제후사용의차이는무엇입니까?

적용 기준과 근거

명시한 출처 판본의 원리를 설명하며 제품별 구현 차이는 별도 확인합니다.

내용 검토 상태: 부분 확인. 출처 확인은 공식 정답이나 학습 성취를 뜻하지 않습니다.

출처별 상세 열람 범위는 위키의 출처 기록에서 확인합니다.

- malloc(3) - Linux man-pages project; 상시 문서; 확인 2026-11-05.
- OSTEP: Segmentation - Remzi H. Arpaci-Dusseau; Andrea C. Arpaci-Dusseau / UW-Madison; 공개판 1.10; 확인 2026-11-05.